Análisis de Complejidad

1. Algoritmo: Ordenamiento de resultados (getSortedResults)

Descripción

Este método pertenece a la clase PlayerStats y se encarga de:

- Copiar los resultados almacenados en una lista enlazada hacia un arreglo
- Ordenarlos utilizando el criterio definido en compareTo (movimientos y tiempo)

El algoritmo de ordenamiento implementado corresponde a un **Bubble Sort**, el cual compara e intercambia elementos en múltiples iteraciones.

Análisis de Complejidad

Tipo de complejidad	Resultado	Justificación
Temporal	$O(n^2)$	Se utilizan dos ciclos anidados que recorren el arreglo completo, realizando comparaciones entre cada par de elementos
Espacial	$O(n)$	Se crea un arreglo auxiliar del mismo tamaño que la lista original

Explicación

El algoritmo recorre el arreglo múltiples veces, comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto. Este proceso se repite hasta que el arreglo queda completamente ordenado.

Debido a esto, el número de operaciones crece proporcionalmente al cuadrado del número de elementos:

```
for (int a = 0; a < size - 1; a++) {
    for (int b = 0; b < size - a - 1; b++) {
        if (copy[b].compareTo(copy[b + 1]) > 0) {
            GameResult temp = copy[b];
            copy[b] = copy[b + 1];
            copy[b + 1] = temp;
        }
    }
}
```

Se evidencia el uso de dos ciclos anidados, lo que genera una complejidad cuadrática O(n²)”

2. Algoritmo: Movimiento del jugador (move(Direction direction))

Descripción

Este método pertenece a la clase GameController y es el encargado de gestionar la lógica principal del juego. Sus funciones incluyen:

- Validar si el movimiento es permitido
- Detectar colisiones con paredes o cajas
- Mover al jugador dentro del tablero
- Empujar cajas cuando sea necesario
- Actualizar el estado del juego
- Registrar estados visitados para evitar ciclos
- Guardar el estado anterior en una pila (undo)

Análisis de Complejidad

Tipo de complejidad	Resultado	Justificación
Temporal	O(1)	Todas las operaciones realizadas son constantes: accesos a matriz, validaciones y actualizaciones
Espacial	O(1)	No se crean estructuras adicionales que dependan del tamaño de entrada

Explicación

A pesar de que el método es extenso en términos de código, no contiene ciclos ni recursividad, todas las operaciones realizadas (como acceso a celdas del tablero, validaciones y modificaciones del estado) se ejecutan en tiempo constante

El uso de estructuras como la tabla de transposición (visitedStates) también opera en tiempo constante promedio

Conclusiones

- El algoritmo más costoso del sistema es el de ordenamiento (getSortedResults), debido al uso de un enfoque cuadrático

- El método move presenta una complejidad constante, lo que lo hace eficiente para la ejecución en tiempo real del juego